Compiling C for a Hint-Based VLIW: An LLVM Back End for STARBUG on RISC-V Wally

Midterm Report

Leo Marek, Gregory Chekler*, Jing Jing Chen*, Elliot Chae*, Ethan Carr*, and Ray Simar
Department of Electrical and Computer Engineering, Rice University, Houston, TX, USA
{lnm7, gsc8, jc165, ejc12, ehc6, ray.simar}@rice.edu *Equal contribution

***Abstract*—STARBUG is a four-lane VLIW extension to the CORE-V Wally RISC-V core in which each bundle is announced by the compressed instruction `c.li x0,` N, an architectural no-op. The same binary therefore runs unmodified on a stock RV32C core and in parallel on STARBUG. Our earlier evaluation relied on hand-scheduled assembly, which left open whether a compiler can target the design at all. This report describes midterm progress on an LLVM back end that forms bundles automatically from ordinary C. It combines a post-register-allocation packetizer that keeps every bundle correct under both parallel and sequential execution, an I-cache-line-aware layout pass, and a static bundle verifier. In RTL simulation of five CMSIS-DSP kernels, compiled code reaches a 1.40× geometric-mean speedup over scalar code versus 1.47× for hand scheduling, recovering 95.6% of the hand-scheduled gain. No compiled binary contains an illegal bundle, whereas four of ten hand-scheduled binaries do.**

***Index Terms*—RISC-V, VLIW, LLVM, instruction scheduling, DSP, CORE-V Wally**

I. Introduction

Embedded DSP workloads need high instruction throughput under tight power and area budgets. Superscalar and out-of-order cores find instruction-level parallelism (ILP) dynamically, but at a hardware cost that is hard to justify in a small core. VLIW machines move scheduling to the compiler [1], but traditional VLIW encodings break binary compatibility, which has limited their adoption in the RISC-V ecosystem.

STARBUG takes a hybrid approach. A RISC-V HINT, the 16-bit `c.li x0,` N with $1 \le N \le 4$ [2], tells a modified CORE-V Wally core [3], [4] that the next N instructions are independent and may issue together. A standard core sees a write to `x0` and treats it as a no-op. Last semester we modeled the bundle semantics in Google's MPACT simulator, implemented four-lane issue in Wally RTL, extended the floating-point (FP) datapath to all lanes, and synthesized the design for a Xilinx VCU108 FPGA. Hand-scheduled integer kernels ran 1.24–1.51× faster than scalar code.

Reviewers of that work raised one main objection: for an ISA extension whose selling point is that existing toolchains keep working, hand-scheduled kernels show the hardware's upper bound but say nothing about whether a compiler can produce valid bundles. This semester's objective is to answer that question. Our goals are to (i) build an LLVM [5] back end that emits STARBUG bundles from unmodified C, (ii) measure it against hand scheduling on the same RTL, and (iii) make bundle legality checkable, since the hardware does not check it. This report covers the architecture briefly and focuses on the compiler.

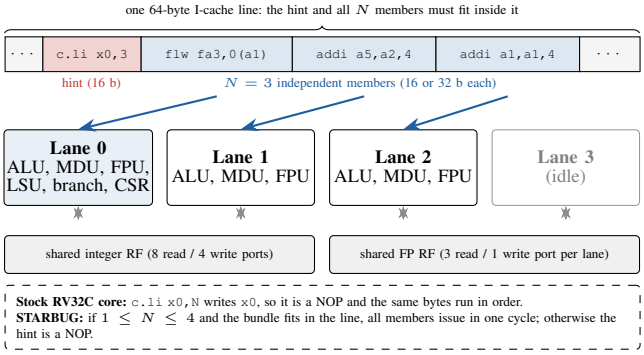


Fig. 1. STARBUG bundle encoding and lane structure. The three-instruction bundle is taken from the hand-scheduled biquad kernel; its single memory operation is placed in lane 0.

II. Methods

A. STARBUG Architecture Overview

Fig. 1 summarizes the design. The instruction fetch unit (IFU) recognizes the hint before decompression, extracts up to four 16- or 32-bit payload instructions from the current I-cache line, and dispatches them to four lanes that share a widened integer register file (8 read / 4 write ports) and a widened FP register file. Lane 0 is a full Wally execution unit. Worker lanes 1–3 handle integer, multiply, and FP arithmetic, but their branch-redirect and load/store paths are disconnected. The single load/store unit (LSU) therefore limits each bundle to one memory operation, which must be in lane 0. The fetch unit may also *decline* a bundle: if $N > 4$, if the bundle crosses a 64-byte I-cache line, or if a branch lands inside it. In that case the hint retires as a no-op and the payload runs sequentially. To save area there is no interlock: an intra-bundle dependence or a worker-lane memory operation silently produces a wrong result, not a fault.

B. Compiler Obligations

Because a bundle is a request that the hardware may refuse, the compiler must satisfy three obligations that a conventional VLIW compiler does not face:

TABLE I
REGISTER HAZARDS BETWEEN MEMBERS OF ONE BUNDLE

Hazard	Parallel issue	Sequential fallback (if reordered)
RAW	unsafe (no same-cycle bypass)	unsafe
WAW	unsafe (no defined winner)	unsafe
WAR	safe (all lanes read first)	unsafe once order changes

1) *Dual correctness.* Every bundle must be correct both as a parallel issue group and as a sequential instruction stream, since declined bundles and stock cores execute the bytes in order (Table I).
2) *Lane legality without a safety net.* Memory, control-flow, CSR, and PC-relative instructions must be in lane 0, with at most one memory operation per bundle.
3) *Placement.* A bundle exists only if it fits within one I-cache line, so whether it is used depends on where the instructions end up in the final binary. Placement constraints of this kind are otherwise found only in explicitly parallel encodings such as IA-64 [6] and the TI C6x [7].

C. LLVM Back End

The back end is a fork of LLVM targeting `riscv32` with `-mcpu=starbug-vliw`. Fig. 2(a) shows where its passes run in the code generator.

Packetizer. Bundles are formed after register allocation, because legality depends on physical registers. RAW and WAW conflicts always end a packet. WAR is the subtle case: it is harmless in parallel, but becomes illegal if the packetizer reorders instructions. We therefore accept WAR pairs only while the packet still follows the original program order. Instructions are classified by opcode and by `mayLoad`/`mayStore` flags rather than by mnemonic, and FP operations may use any lane, matching the RTL. An unrecognized instruction defaults to lane 0, which is always safe.

Bundle layout pass. This pass runs last, after compression, branch relaxation, and pseudo-instruction expansion, because byte offsets are not final until then. It tracks the emitted size of every instruction, including branches that the assembler will compress, and splits any bundle that would cross a 64-byte line into two bundles at the boundary (Fig. 2(b)). Splitting never moves an instruction, adds no padding, and preserves program order. Any subset of an independent set is still independent, so both halves remain legal.

Unroll policy. Spills are memory operations that compete for the single lane-0 LSU, so we cap unrolling at a factor of 8 (maximum 16), the best point in our sweep.

D. Verification and Evaluation

Bundle verifier. A standalone checker reconstructs every bundle from the final ELF encodings. It rejects intra-bundle RAW/WAW conflicts, illegal lane assignments, and hints with $N > 4$. The checker uses the same hardware contract, read directly from the RTL, as the back end.

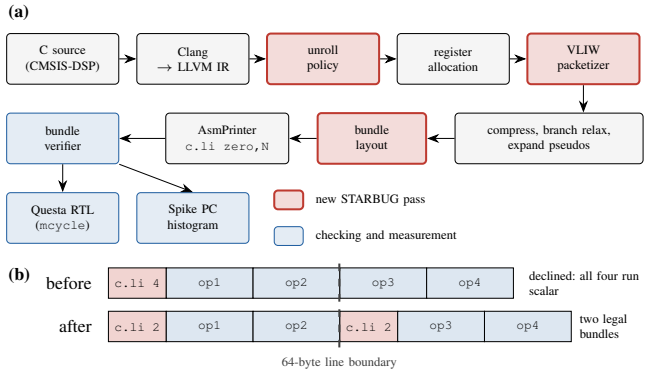


Fig. 2. (a) STARBUG passes (red) in the LLVM RISC-V code generator and the checking flow (blue). (b) The layout pass splits a bundle that would cross an I-cache line; no instruction moves and no padding is added.

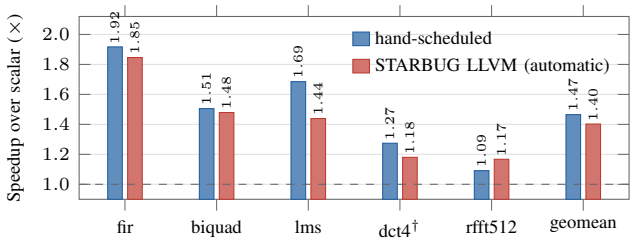


Fig. 3. Kernel-region speedup over the GCC `-O3` scalar build (Questa RTL, layout pass enabled). Dashed line: scalar. †`dct4` misses its SNR threshold in every build, including clang without the packetizer (Sec. IV).

Workloads and measurement. We use five CMSIS-DSP [8] float32 kernels (FIR, biquad, LMS, DCT-IV, real FFT) and build each one three ways: GCC `-O3` scalar (`rv32gc`), hand-scheduled assembly, and STARBUG LLVM. We report cycles in the kernel region (`mcycle`) from full-RTL Questa simulation, including caches and LSU contention. For bundle coverage and ILP, we join Spike [9] PC histograms with the static bundle map. This is valid because the hint is a no-op on Spike. Each kernel also checks its output signal-to-noise ratio (SNR) against a golden reference.

III. RESULTS (PARTIAL)

Speedup. Fig. 3 compares compiled and hand-scheduled code. Compiled code reaches a 1.402× geometric mean over scalar, versus 1.465× for hand scheduling, so the compiler recovers 95.6% of the hand-scheduled speedup with no manual annotation. On `rfft512` the compiled build is 6.9% *faster* than the hand-scheduled one. The hand-written hints cover only the code path taken by 2048-point transforms, so at 512 points the hand-scheduled hot function `arm_radix8_butterfly_f32` runs entirely unbundled (0.0% coverage, ILP 1.00). The compiler covers both paths (81.4% coverage, ILP 2.32). The largest gap is `lms`, where the compiler reaches 85% of the hand speedup.

Legality. Across all twenty binaries, the verifier finds illegal bundles in 4 of 10 hand-scheduled binaries and in none of the 10 compiled ones. The defects include a branch in a worker

TABLE II
STATICALLY BUNDLED INSTRUCTIONS BEFORE AND AFTER EACH BACK-END FIX, AND SPEEDUP FROM THE LAYOUT PASS

Fix	Kernel	Before	After
Debug instructions ignored (`-g`)	biquad	40	144
FP allowed in worker lanes	biquad	69	155
Intra-bundle WAR accepted	cfft_radix8	195	237
Line-aware layout (speedup)	5-kernel geomean	—	+1.98%

lane on a live path in `arm_dct4_f32`. Because worker lanes cannot redirect the PC, the branch falls through into an unrolled body and causes a silent out-of-bounds access, which the kernel's output check happens to miss. A second defect is an intra-bundle RAW pair in `arm_bitreversal_16`.

Effect of individual fixes. Table II isolates the main back-end fixes, measured in statically bundled instructions per kernel. The largest effect came from debug information. Each `DBG_VALUE` ended the packet in progress, so building with `-g` removed about three quarters of all bundling. Hand-written assembly was unaffected, so this defect lowered only the compiler's numbers. The layout pass adds 1.98% geometric-mean speedup (up to 4.4% on biquad). Disassembly with hints removed shows that the pass changes only PC-relative immediates.

IV. DISCUSSION (PARTIAL)

The midterm results suggest that STARBUG can be targeted by a compiler. Automatic bundling reaches most of the hand-scheduled performance, covers code that hand scheduling missed, and is the only producer in our study whose output passes the legality check. The remaining gap has two known causes. First, bundles cannot span basic blocks. In biquad, the pointer updates that the hand-written code folds into FP bundles sit past a branch in compiled code (ILP 2.14 versus 2.61), which global scheduling [10], [11] would address. Second, there is no modulo scheduling [12]. FIR and LMS are limited by the single LSU, so the best schedule pairs each load with the previous tap's multiply-add. That pairing requires software pipelining, and more unrolling does not help.

Two findings concern the toolchain rather than the architecture. Linker relaxation removes bytes after the layout pass has run, which invalidates its offsets. Disabling relaxation makes the pass more effective, but costs FIR about 6% on its own. Layout-sensitive hints therefore need support across the whole toolchain, including the linker. Also, an estimate that line-crossing bundles cost FIR 20% of runtime proved wrong: removing most of them gained 0.34%. Decline rates are not a performance model, so we report RTL cycles.

Several threats to validity remain, and they set our plan for the rest of the semester. Correctness currently rests on per-kernel SNR checks and the static verifier. LMS and rfft pass by only 1 dB, so we will add lockstep comparison against Spike. The scalar baseline uses GCC rather than clang, so part of the speedup may come from the different compiler; a clang scalar build will isolate the effect of bundling. `dct4_512` misses its 118 dB threshold in every configuration, including clang with no packetizer (110 dB), because the threshold was calibrated against GCC's floating-point contraction. We will recalibrate it. Our FPGA area figures appear to compare integer-only and FP-enabled STARBUG rather than stock Wally, so we will re-synthesize against an unmodified `rv32gc` configuration. Finally, the workload set is narrow. We will extend the evaluation to 13 integer and control-heavy kernels and to the five benchmark variants not yet measured with the layout pass, and we will replace the static branch-size guess in the layout pass with an iterative branch-distance model.

ACKNOWLEDGEMENTS

We thank David Harris, James Stine, and the OpenHW Group for the open-source CORE-V Wally core, the developers of Google MPACT and Spike, and Rice University ECE for support. Generative AI tools were used to help write code, debug, and draft and edit this report. The authors reviewed all such content and take full responsibility for it.

CREDIT AUTHOR STATEMENT

Roles follow the CRediT taxonomy [13]. **Leo Marek:** Conceptualization, Methodology, Software (LLVM back end, verifier), Validation, Formal Analysis, Investigation, Visualization, Writing – Original Draft, Writing – Review & Editing. **Gregory Chekler, Jing Jing Chen, Elliot Chae, Ethan Carr:** Methodology, Validation, Investigation, Writing – Review & Editing. **Ray Simar:** Conceptualization, Supervision, Resources.

REFERENCES

[1] J. A. Fisher, "Very long instruction word architectures and the ELI-512," in *Proc. 10th Annu. Int. Symp. Computer Architecture (ISCA)*, 1983, pp. 140–150.

[2] *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V International, 2024. [Online]. Available: https://riscv.org/specifications/

[3] OpenHW Group, "CORE-V Wally: Configurable RISC-V processor," GitHub repository, 2026, accessed Oct. 2026. [Online]. Available: https://github.com/openhwgroup/cvw

[4] D. Harris, J. Stine, S. Harris, and R. Thompson, *RISC-V System-on-Chip Design*. Cambridge, MA, USA: Morgan Kaufmann, 2025.

[5] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[6] M. S. Schlansker and B. R. Rau, "EPIC: Explicitly parallel instruction computing," *Computer*, vol. 33, no. 2, pp. 37–45, Feb. 2000.

[7] *TMS320C6000 Technical Brief*, Texas Instruments, Feb. 1999, lit. no. SPRU197D.

[8] Arm Limited, "CMSIS-DSP software library," GitHub repository, 2025. [Online]. Available: https://github.com/ARM-software/CMSIS-DSP

[9] RISC-V International, "Spike RISC-V ISA simulator," GitHub repository, 2026. [Online]. Available: https://github.com/riscv-software-src/riscv-isa-sim

[10] D. Bernstein and M. Rodeh, "Global instruction scheduling for superscalar machines," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 1991, pp. 241–255.

[11] W.-M. W. Hwu *et al.*, "The superblock: An effective technique for VLIW and superscalar compilation," *J. Supercomputing*, vol. 7, no. 1–2, pp. 229–248, 1993.

[12] B. R. Rau, "Iterative modulo scheduling: An algorithm for software pipelining loops," in *Proc. 27th Annu. Int. Symp. Microarchitecture (MICRO)*, 1994, pp. 63–74.

[13] NISO, "CRediT: Contributor roles taxonomy," 2022. [Online]. Available: https://credit.niso.org/